

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных систем и технологий
Кафедра «Информационные системы»
Дисциплина «Тестирование программного обеспечения»

Лабораторная работа № 5

Выполнил:
студент гр. ПИБд - 31
Белянин Н.Н.
Проверила:
Островская С.Ф.

Ульяновск, 2025г.

1. Задание.

1. Основываясь на ранее разработанной тестовой документации разработать автотесты для проверки функциональности программной системы по варианту в соответствии с требованиями:

- a. Автотесты необходимо проектировать, исходя из свойств сопровождаемости, надежности, структурированности.
- b. Автотесты можно реализовать в форме отдельного приложения, в виде модульных тестов.
- c. Автотесты должны эмулировать поведение пользователя при взаимодействии с элементами пользовательского интерфейса. Обязательно реализовать контроль результатов выполнения сценария.

2. В отчет по лабораторной работе включить:

- a. Титульный лист.
- b. Задание, вариант.
- c. Описание тестируемого приложения.
- d. Описание реализованных автотестов: инструменты, подходы.
- e. Код автотестов.
- f. Отчет о тестировании (выполненные тест-кейсы, результат, выявленные дефекты).
- g. Выводы по работе.
- h. Список использованных источников.

3. Оформить и защитить отчет.

2. Вариант.

БЕЛЯНИН	НИКИТА	НИКОЛАЕВИЧ	Auto.ru
---------	--------	------------	---------

3. Описание тестируемого приложения.

Auto.ru — это онлайн-платформа, предназначенная для покупки, продажи автомобилей, а также для аренды и обмена транспортными средствами. Приложение предоставляет пользователям удобный интерфейс для поиска машин по различным критериям (марка, модель, цена, год выпуска,

пробег, цвет автомобиля, тип кузова). Платформа включает такие основные функциональные возможности, как:

- **Просмотр каталога:** запущенное приложение предоставляет каталог автомобилей для просмотра и изучения. Лента формируется на основе отзывов, рейтинга и города, в котором проживает пользователь, от самых популярных до менее известных. Также для пользователя предоставляют каталог разделов, где можно получить нужную информацию, а также есть раздел предложенных продуктов и связь техническая поддержка.
- **Фильтрация автомобилей:** фильтрация, сортировка по множеству параметров, включая цену, марку, модель, год выпуска, пробег и местоположение.
- **Поиск автомобилей:** поиск автомобилей осуществляется через поисковые строки по маркам, которые по совместительству являются названиями объявлений.
- **Проверка истории автомобиля:** интеграция с сервисами для проверки VIN-кода автомобиля и получения информации о его предыдущих владельцах и авариях.
- **Отзывы и рейтинги:** система оценки продавцов и автомобилей на основе отзывов пользователей, а также для пользователей предоставляется возможность.

4. Описание реализованных автотестов.

Инструменты:

1. **Python.** Для реализации тестов был выбран язык программирования Python, который является одним из любимых в командах тестирования. Данный язык программирования обладает множеством расширений, фреймворкам и функций, которые помогают автоматизировать работу с тестированием сайта или мобильного приложения. Python является прекрасным выбором для автоматизированного тестирования.

2. **PyTest.** В реализации тестов используется фреймворк для автоматизированного тестирования на Python, который предоставляет простой синтаксис и мощные возможности для решения разнообразных задач. Благодаря его гибкости пользователи могут выполнять очень необычные тесты, в соответствии с функционалом. Благодаря данному фреймворку, можно легко и быстро написать тесты для приложения.
3. **Selenium WebDriver.** Это программная библиотека для управления браузерами. WebDriver представляет собой драйверы для различных браузеров и клиентские библиотеки на разных языках программирования, предназначенные для управления ими. Использование Selenium WebDriver сводится к автоматизации работы с браузером, что делает его незаменимым инструментом для тестирования веб-приложений. Данная библиотека позволяет интерактивно проходить по всему приложению, выполняя весь задуманный функционал.

Подходы:

1. **POM – PageObjectModel.** Подход, позволяющий абстрагировать представление тестируемого приложения от сценариев тестирования. Делается это с целью сокращения трудоемкости актуализации тестов в случае изменения вёрстки веб-страниц приложения, изменения адресации элементов.
2. **Сценарный подход.** Это процесс проверки корректности работы программного обеспечения путём имитации действий конечного пользователя. Вместо того чтобы тестировать отдельные функции или модули ПО, тестировщик выполняет целый ряд действий, которые пользователь мог бы совершить в реальных условиях использования продукта.

5. Код автотестов.

Selenium используется для написания тестовых сценариев, которые задуманы функционалом. Для написания тестов используется модуль Python

PyTest. Существует несколько способов поиска элементов на странице, таких как поиск по XPath, поиск по классу, по ID. В данной лабораторной преимущественно использовался поиск по XPath.

При разработке автотестов был использован паттерн проектирования объектов страницы. Объект страницы представляет пользовательский интерфейс в веб-приложении, с которым взаимодействует тест. Преимущества включают создание многократно используемого кода, уменьшение дублирования и минимальные изменения при изменении интерфейса пользователя. Это обеспечивает удобную структуру проекта.

Исходный код:

```
Base_page.py
from selenium.common import NoSuchElementException
from selenium.webdriver.remote.webdriver import WebDriver
from selenium.webdriver.remote.webelement import WebElement
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as ec
```

```
class BasePage:
    # Драйвер для работы с веб-страницами
    def __init__(self, driver: WebDriver):
        self._driver = driver

    def _scroll_into_view(self, locator: tuple):
        element = self._find(locator)

self._driver.execute_script("arguments[0].scrollIntoView(true);",
element)

    # Поиск и возврат одного элемента на веб-странице с
    # помощью переданного локатора
    def _find(self, locator: tuple) -> WebElement:
        return self._driver.find_element(*locator)

    # Ввод текста в элемент на веб-странице
    def _type(self, locator: tuple, text: str, time: int = 10):
        # Прокручиваем элемент в видимую область
        self._scroll_into_view(locator)
        self._wait_until_element_is_visible(locator, time)
        self._find(locator).send_keys(text)

    # выполнение клика по элементу на веб-странице
    def _click(self, locator: tuple, time: int = 10):
        self._wait_until_element_is_visible(locator, time)
        self._find(locator).click()

    # ожидание появления элемента на веб-странице до тех пор,
    # пока он не станет видимым
    def _wait_until_element_is_visible(self, locator: tuple, time: int =
10):
        wait = WebDriverWait(self._driver, time)
        wait.until(ec.visibility_of_element_located(locator))

    # открытие указанного URL веб-страницы
    def _open_url(self, url: str):
        self._driver.get(url)

    # получение текстового содержимого элемента, найденного по
    # указанному локатору.
    def _get_text(self, locator: tuple, time: int = 3) -> str:
```

```
self._wait_until_element_is_visible(locator, time)
return self._find(locator).text.encode("utf-8").decode("utf-8")
```

```
home_page.py
import time
```

```
from selenium.webdriver.common.by import By
from selenium.webdriver.remote.webdriver import WebDriver
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as ec
from selenium.webdriver.common.keys import Keys
```

```
from base_page import BasePage
```

```
class HomePage(BasePage):
```

```
    # Сайт
    url = "https://auto.ru/"
```

```
    __history_link = "https://auto.ru/history/"
    __catalog_link = "https://auto.ru/catalog/cars/"
```

```
    # Поля для поиска
    __input_search = (By.XPATH, "//*[@contains(@class,
'SearchLineSuggest__input')]/input")
    __search_results = (By.XPATH,
"//*[@contains(@class,'ListingHead__content')]/h1")
    __empty_search = (By.XPATH, "//*[@contains(@class,
'Index')]/div")
```

```
    # Фильтрация
    __filter_price_from = (By.XPATH, "//*[@label[contains(@class,
'TextInput TextInput_size_1 TextInput_has-clear TextInput_has-
placeholder TextInput_placeholderShowWithValue')]/input")
    __filter_price_to = (By.XPATH, "//*[@label[contains(@class,
'TextInput TextInput_size_1 TextInput_has-clear TextInput_has-
placeholder TextInput_placeholderShowWithValue')]/input")
    __filter_button = (By.XPATH,
"//button[contains(@class,'Button2-DDOYo
Button2_themeType_primary-bnQga Button2_size_m-btc12
Button2_color_black-NJF2l CatalogFilters__button-lGshV')]"
```

```
    # Каталог
    __catalog_section = (By.XPATH, "//*[@contains(@class,
'Index')]"")
    __catalog_first_page = (By.XPATH,
"//*[@contains(@class,'BaseApp__content AppCatalog__content-
gicEX')]"")
    # Проверка авто по VIN
```

```

__vin_input = (By.XPATH, "///div[contains(@class,
'VinCheckInput3__controls-EZidM')]/input")
__vin_check_button = (By.XPATH,
"//button[contains(@class,'Button2-DDOYo
Button2_themeType_primary-bnQga Button2_size_l-giThR
Button2_color_lightStatic-hPynB')]")
__vin_result = (By.XPATH,
"//div[contains(@class,'VinReportPreviewDesktop__mmm')]/div")

def __init__(self, driver: WebDriver):
    super().__init__(driver)

# Перейти на страницу истории автомобиля по VIN
def go_to_history(self):
    super().__open_url(self.__history_link)

def go_to_catalog(self):
    super().__open_url(self.__catalog_link)

# Поиск автомобиля
def execute_search(self, search_text: str):
    # Кликаем на поле поиска через super()
    super().__click(self.__input_search)
    time.sleep(4)

# Заполняем поле поисковым текстом через super()
super().__type(self.__input_search, search_text)
time.sleep(2) # Дополнительная задержка перед отправкой,
если необходимо

# Вместо клика по кнопке поиска, отправляем форму с
помощью Enter
element = self._find(self.__input_search)
element.send_keys(Keys.RETURN) # Нажимаем Enter для
отправки формы
time.sleep(4)

# Фильтрация по цене
def execute_filter_hours_strong(self, min_price: int, max_price:
int):
    min_price_input = self._find(self.__filter_price_from) # Найти
поле ввода
    min_price_input.clear()
    min_price_input.send_keys(str(min_price)) # Ввести
минимальную цену
    time.sleep(1)

    max_price_input = self._find(self.__filter_price_to) # Поле
для максимальной цены
    max_price_input.clear()
    max_price_input.send_keys(str(max_price)) # Ввести
максимальную цену
    time.sleep(1)

    super().__click(self.__filter_button) # Нажать кнопку
фильтрации
    time.sleep(2)

# Открытие каталога
def open_catalog(self):
    super().__click(self.__catalog_section)
    time.sleep(2)

# Проверка авто по VIN
def check_vin(self, vin: str):
    super().__type(self.__vin_input, vin)
    time.sleep(1)
    super().__click(self.__vin_check_button)
    time.sleep(2)

# Открыть сайт
def open(self):
    super().__open_url(self.url)

# Получить первый результат поиска
def get_first_search_result(self) -> str:
    WebDriverWait(self._driver, 10).until(
        ec.visibility_of_element_located(self.__search_results)
    )

```

```

return super().__get_text(self.__search_results, time=3)

# Проверка пустого поиска
def get_empty_search(self) -> str:
    WebDriverWait(self._driver, 10).until(
        ec.visibility_of_element_located(self.__empty_search)
    )
    return super().__get_text(self.__empty_search, time=3)

# Получить первый результат после фильтрации
def get_first_filtered_result(self) -> str:
    return super().__get_text(self.__filtered_results, time=3)

# Проверка загрузки первой страницы каталога
def get_first_catalog_result(self) -> str:
    WebDriverWait(self._driver, 10).until(
        ec.visibility_of_element_located(self.__catalog_first_page)
    )
    return super().__get_text(self.__catalog_first_page, time=3)

# Получить результат проверки по VIN
def get_vin_result(self) -> str:
    WebDriverWait(self._driver, 10).until(
        ec.visibility_of_element_located(self.__vin_result)
    )
    return super().__get_text(self.__vin_result, time=3)

test.py
import pytest
import undetected_chromedriver as uc
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as
ChromeService
from webdriver_manager.chrome import ChromeDriverManager

from home_page import HomePage

class TestCatalog:
    def test_catalog_first_page(self):
        driver =
webdriver.Chrome(service=ChromeService(ChromeDriverManager()
.install()))
        driver.maximize_window()
        auto_page = HomePage(driver)
        auto_page.open()

        auto_page.go_to_catalog() # Теперь просто открываем
ссылку

        driver.quit()

class TestFilter:
    def test_filter_by_hours_strong(self):
        driver =
webdriver.Chrome(service=ChromeService(ChromeDriverManager()
.install()))
        driver.maximize_window()
        auto_page = HomePage(driver)
        auto_page.open()
        auto_page.go_to_catalog()
        auto_page.execute_filter_hours_strong(min_price=90,
max_price=150)
        #assert auto_page.check_price_filter(min_price=500000,
max_price=1500000)
        driver.quit()

class TestVinCheck:
    @pytest.mark.parametrize('vin_code', [('K804PT73')])
    def test_vin_check(self, vin_code):
        driver =
webdriver.Chrome(service=ChromeService(ChromeDriverManager()
.install()))
        driver.maximize_window()
        auto_page = HomePage(driver)
        auto_page.open()
        # Перейти на страницу истории автомобиля
        auto_page.go_to_history()
        auto_page.check_vin(vin_code)
        # assets auto_page.get_vin_result().find(vin_code) != 1

```

```

driver.quit()

class TestSearch:
    @pytest.mark.parametrize('search_text', [('BMW')])
    def test_search_positive(self, search_text):
        driver =
webdriver.Chrome(service=ChromeService(ChromeDriverManager()
.install()))
        driver.maximize_window()
        auto_page = HomePage(driver)
        auto_page.open()
        auto_page.execute_search(search_text)
        assert auto_page.get_first_search_result().find(search_text) != -

1

driver.quit()

@pytest.mark.parametrize('search_text', [('bfdbfdbfd')])
def test_search_negative(self, search_text):
    driver =
webdriver.Chrome(service=ChromeService(ChromeDriverManager()
.install()))
    driver.maximize_window()
    auto_page = HomePage(driver)
    auto_page.open()
    auto_page.execute_search(search_text)
    assert auto_page.get_empty_search().find(search_text) == -1
    driver.quit()

```

```

PS E:\ТПО\Лабораторные работы\Лабораторная работа 5\AutoTest> pytest -s test.py
===== test session starts =====
platform win32 -- Python 3.12.6, pytest-8.3.4, pluggy-1.5.0
rootdir: E:\ТПО\Лабораторные работы\Лабораторная работа 5\AutoTest
collected 5 items

test.py
DevTools listening on ws://127.0.0.1:62626/devtools/browser/c853b1d8-1511-4ab9-9acb-28a845c1e3dd
*
DevTools listening on ws://127.0.0.1:62687/devtools/browser/afc610bb-9060-4ee6-9922-36a0e8d42134
*
DevTools listening on ws://127.0.0.1:62753/devtools/browser/15f2618c-051d-4160-8d8e-c319c30889e2
*
DevTools listening on ws://127.0.0.1:62817/devtools/browser/a4acec18-b266-4162-b877-d944b81fff2c
[15148:9028:0211/123146.850:ERROR:socket_manager.cc(147)] Failed to resolve address for stun3.l.google.com., errorcode: -105
[15148:9028:0211/123146.851:ERROR:socket_manager.cc(147)] Failed to resolve address for stun3.l.google.com., errorcode: -105
[15148:9028:0211/123146.851:ERROR:socket_manager.cc(147)] Failed to resolve address for stun3.l.google.com., errorcode: -105
[15148:9028:0211/123146.851:ERROR:socket_manager.cc(147)] Failed to resolve address for stun4.l.google.com., errorcode: -105
[15148:9028:0211/123146.851:ERROR:socket_manager.cc(147)] Failed to resolve address for stun4.l.google.com., errorcode: -105
[15148:9028:0211/123146.851:ERROR:socket_manager.cc(147)] Failed to resolve address for stun4.l.google.com., errorcode: -105
[17512:11568:0211/123147.335:ERROR:command_buffer_proxy_impl.cc(331)] GPU state invalid after WaitForGetOffsetInRange.
*
DevTools listening on ws://127.0.0.1:62923/devtools/browser/03bba122-4c38-4bb3-8f8a-a487d59fc4bf
*
===== 5 passed in 130.72s (0:02:10) =====
PS E:\ТПО\Лабораторные работы\Лабораторная работа 5\AutoTest>

```

Рисунок 1. Успешное выполнение тестирования

6. Отчет о тестировании.

Выполненные тест-кейсы:

ID	Тест-кейс	Описание (Тип)	Предуслови я	Шаги	Ожидаемый результат	Результат
1	Просмотр каталога.	После запуска веб-приложения, пользователь должен попадать на главный экран каталога.	Доступ в интернет и соединение с сетью.	Зайти на сайт, либо запустить веб-приложение	Пользователь попадает на главный экран каталога.	Сайт успешно открывается, и пользователь попадает на каталог.
2	Поиск по строке (положительный).	Пользователь вводит в поисковую строку текст и по тексту запроса находит необходимый автомобиль. В конечном результате пользователь находит нужный автомобиль.	Доступ в интернет и соединение с сетью.	1.Перейти в поисковую строку 2.Ввести нужный запрос. 3.Получить результат	Пользователь получает нужную информацию по запросу.	Сайт успешно обрабатывает запрос.

3	Поиск по строке (негативный).	Пользователь вводит в поисковую строку текст и по тексту запроса не находит необходимый автомобиль. Он получает сообщение о том, что "Ничего не найдено".	Доступ в интернет и соединение с сетью.	1.Перейти в поисковую строку 2.Ввести нужный запрос. 3.Получить результат	Пользователь получает сообщение, что ничего не найдено по запросу.	Сайт выводит сообщение "Ничего не найдено".
4	Фильтрация автомобилей.	Пользователь устанавливает параметры фильтрации и по ним отбирает автомобили, которые соответствуют его требованиям.	Доступ в интернет и соединение с сетью.	1. Пользователь переходит на каталог 2. В параметрах фильтрации устанавливает необходимые фильтры 3. Нажимает на кнопку "Показать"	Пользователь получает отфильтрованный список автомобилей.	Сайт отфильтровывает список автомобилей и отображает их.
5	Поиск автомобиля по VIN-коду.	Пользователь в разделе "Отчёты" вписывает в поисковую строку для VIN-кода гос. номер машины, либо VIN-код и ему предоставляется отчёт.	Доступ в интернет и соединение с сетью.	1. Пользователь переходит в раздел "Отчёты". 2. Пользователь в строке поиска по VIN вводит гос.номер/VIN-код. 3. Нажимает на кнопку "Проверить".	Пользователь получает отчёт по автомобилю.	Сайт показывает отчёт по автомобилю.

Результаты: В ходе выполнения тестирования, ошибок не происходило, все тестируемые элементы, и функционал успешно выполнил заявленные задачи.

Выявленные дефекты: Критичных дефектов обнаружено не было, которые крайне негативно повлияли бы на работу программы.

7. Вывод.

В ходе выполнения лабораторной работы была приобретена практика в написании автотестов, применяемых в разработке программных продуктов. Была изучена концепция автоматизированного тестирования на основе библиотек, расширений и фреймворков. Была изучена документация работы функционала тестирования. Автоматизация помогает нам выполнять задачи быстрее, не тратя личное время, с меньшими ошибками. Все знания закреплены практически – было реализованы автотесты для заранее выбранного веб-приложения.

8. Список использованных источников.

1. Документация сайта auto.ru.
2. Официальный сайт auto.ru.
3. Лекционный материал для углубленного понимания тестирования (ТестируемоеПО 3.pptx).
4. Дополнительные методические материалы (МетодичкаТПО.pdf).
5. Тестирование и Качество ПО url: Software-Testing.Ru.
6. Практическое руководство по нагрузочному тестированию.
7. OWASP Testing Guide – руководство по тестированию безопасности.